

Intro COBOL

El lenguaje común orientado a los negocios (COBOL) es un lenguaje de programación compilado de alto nivel desarrollado específicamente para satisfacer las necesidades de procesamiento de datos empresariales

Uno de los grandes puntos fuertes de COBOL es su fuerte soporte para cálculos decimales de punto fijo de gran precisión, una característica no necesariamente nativa de muchos lenguajes de programación tradicionales. Esta capacidad ayudó a diferenciar a COBOL e impulsar su adopción por parte de muchas instituciones financieras grandes.

Fue uno de los primeros lenguajes de programación normalizados por el Instituto Nacional Estadounidense de Estándares (ANSI) y la Organización Internacional de Normalización (ISO).

Historia

COBOL fue desarrollado por un conjunto de organizaciones gubernamentales y empresariales llamado Conferencia sobre Lenguajes de Sistemas de Datos (CODASYL).

Es un derivado en parte de FLOW-MATIC, un lenguaje creado por la pionera de la informática, la Dra. Grace Hopper

La primera versión del lenguaje de programación COBOL se publicó en 1960 pero fue finalmente estandarizado como lenguaje informático en 1968.

Actualmente es menos popular que otros lenguajes más modernos (como Python, Java y JavaScript), sin embargo sigue siendo una parte funcional y crítica de la infraestructura tecnológica mundial, especialmente para instituciones bancarias, compañías de seguros y organismos gubernamentales.

Datos

La sintaxis de COBOL es autodocumentada y casi autoexplicativa, con énfasis en la verbosidad (muchas líneas de código para tareas simples) y la legibilidad.

Ventajas:

- Ofrece gran estabilidad y fiabilidad en aplicaciones de misión crítica
- Permite escalar las aplicaciones sin cambios significativos en la base de código
- Ofrece capacidades excepcionales de procesamiento de archivos
- Gracias a las actualizaciones para modernizarlo puede comunicarse con aplicaciones web

- **Estructura**

IDENTIFICATION DIVISION.	
PROGRAM-ID. YourProgramName	
AUTHOR. Your name	
DATE-WRITTEN. YYYYMMDD	
COMMENT. "Short description of the program"	

- Identification division: Asigna al programa un nombre y proporciona otra información de identificación como el autor, la fecha escrita y una breve descripción del propósito del programa.
- Environment Division: especifica el entorno de tiempo de ejecución de un programa y define los recursos de entrada y salida que utilizará.
- Data Division: contiene todas las definiciones de variables, archivos y constantes para el programa.
- Procedure Division: contiene el código ejecutable del programa, que se divide en párrafos y secciones.

info:

<https://www.ibm.com/es-es/think/topics/cobol>

Bubble Sort:

En ambos lenguajes:

1. Crean un vector de 1.000 elementos.
2. Lo inicializan en orden descendente.
3. Ejecutan Bubble Sort.
4. Miden el tiempo del ordenamiento en OneCompiler.
5. Muestran el resultado.

Como el algoritmo es el mismo, ambos realizan aproximadamente la misma cantidad de comparaciones e intercambios (Bubble Sort tiene complejidad temporal $O(n^2)$).

Búsqueda binaria:

- Calcula el **punto medio** del rango actual y compara ese valor con el elemento buscado. Si el elemento coincide con el valor central, la búsqueda finaliza con éxito devolviendo la posición.
- Si el elemento buscado es menor, el algoritmo descarta toda la mitad derecha actualizando el límite superior.
- Si es mayor, descarta la mitad izquierda ajustando el límite inferior hacia el segmento restante.
- Este proceso de división a la mitad se repite iterativa o recursivamente hasta hallar el dato o hasta que los límites se cruzan.
- Al reducir el espacio de búsqueda a la mitad en cada paso, logra una complejidad logarítmica($\log n$), siendo ideal para grandes volúmenes de datos.

Como el algoritmo es el mismo, ambos realizan aproximadamente la misma cantidad de comparaciones e intercambios (Bubble Sort tiene complejidad temporal $O(\log_{base2} n)$)

Ejecución en COBOL

Cuando se ejecuta el programa en COBOL:

- GnuCOBOL no genera un ejecutable tan optimizado como los compiladores COBOL comerciales. Normalmente traduce COBOL a C y después compila ese C.
- Las comparaciones se pudieron realizar haciendo la diferencia entre el tiempo de inicio y final usando ACCEPT en OneCompiler

Si se usase un compilador comercial, el tiempo de ejecución suele ser menor y el consumo de memoria también.

Ejecución en Scala

Cuando se ejecuta el programa en Scala:

- primero se inicia la **Java Virtual Machine (JVM)**;
- el bytecode generado por Scala es interpretado y luego optimizado por el compilador **JIT (Just-In-Time)**;
- durante la ejecución interviene el **Garbage Collector**, encargado de que no hayan memory leaks.

Todo esto agrega una sobrecarga inicial que normalmente hace que programas pequeños tarden un poco más en comenzar.

Sin embargo, en ejecuciones largas el JIT puede optimizar el código mientras el programa se está ejecutando, mejorando el rendimiento.

JIT significa "Just-In-Time Compiler" (compilador "justo a tiempo").

Cuando detecta que una parte del programa se ejecuta muchas veces (un *hot spot*), hace lo siguiente:

1. Toma ese bytecode.
2. Lo compila a **código máquina nativo** para el procesador.
3. A partir de ese momento, ejecuta la versión optimizada en lugar del bytecode.

Conclusiones

COBOL continúa siendo un lenguaje muy utilizado en sistemas empresariales gracias a su estabilidad, fiabilidad y capacidad para procesar grandes volúmenes de datos.

Cada lenguaje está pensado para necesidades distintas, por lo que la elección depende del problema que se quiera resolver.

El rendimiento no depende únicamente del lenguaje, sino también del algoritmo utilizado y del entorno de ejecución.